Relational Databases and SQL II

Laboratórios de Sistemas e Serviços

Mário Antunes

2026

Exercises

Lab 0: Environment Setup

This class uses **SQLite**, **PostgreSQL**, and **Redis**. To avoid installation issues, we recommend using **Docker**.

1. Navigate to the `02_support` folder.
2. Run `docker compose up -d` to start the PostgreSQL and Redis services.
3. For the Python labs, you can use your local environment (if you have the required libraries) or install them via:

   ```
   pip install -r requirements.txt
   ```

Lab 1: "The Need for Speed" (Indexing Impact)

In this lab, you will measure the performance gain of using an index on a large dataset.

1. **Setup:** Run `python3 lab_utils.py` to generate `lab_indexing.db` with 100,000 records.
2. **Task:** Open the database using `sqlite3 lab_indexing.db`.
3. **Measurement:** Turn on the timer and search for a specific name (one that doesn't exist to force a full scan).

   ```
   .timer on
   SELECT * FROM users WHERE name = 'nonexistent';
   ```

4. **Optimization:** Create an index on the `name` column and run the query again.

   ```
   CREATE INDEX idx_users_name ON users(name);
   SELECT * FROM users WHERE name = 'nonexistent';
   ```

5. **Observation:** Compare the execution times. How many times faster was the indexed query?

Lab 2: "The Breach" (Security & Prepared Statements)

In this lab, you will perform a SQL Injection attack and then fix it.

1. **Setup:** Run `python3 lab_utils.py` to ensure `lab_security.db` is ready.
2. **Attack:** Create a small Python script (or use the one in `solutions/`) that uses string concatenation for a login query.
   - **Hint:** Try to log in as `admin` without knowing the password by using `' OR '1'='1` as the username.

   ```
   username = "admin' --"
   query = f"SELECT * FROM accounts WHERE username = '{username}'"
   ```

3. **Fix:** Rewrite the login function to use **Prepared Statements**.
   - **Hint:** Use `?` placeholders in SQLite.

```
cursor.execute("SELECT * FROM accounts WHERE username = ?", (username,))
```

4. **Verification:** Try the attack again on the secure version. Does it still work?

Lab 3: “Portable Power” (Standalone SQL)

Compare the behavior of file-based (SQLite) and in-memory (H2/SQLite) databases.

1. **SQLite (File):** Create a table in a file-based DB, exit, and re-open it.
2. **SQLite (Memory):** Run `sqlite3 :memory:`, create a table, exit, and re-open.
3. **Observation:** What happened to the data in the memory-only database? When would you use this in a real project?

Lab 4: “The Hybrid Arch” (Dockerized SQL + NoSQL)

Implement a simple “Cache-Aside” pattern using PostgreSQL and Redis.

1. **Setup:** Start the environment using `docker compose up -d`.
2. **Implementation:** Create a script that tries to fetch a user from **Redis** first. If it’s a “Miss”, fetch it from **PostgreSQL** and store it in Redis for next time.
3. **Hint (Python):**

```python
import redis, psycopg2
r = redis.Redis(host='localhost', port=6379)
# Try Redis
data = r.get("user:1")
if not data:
    # Fetch from Postgres if Redis is empty
    # ... fetch logic ...
    r.setex("user:1", 60, "result_from_db")
```

4. **Measurement:** Measure the time for the first request (Cache Miss) vs. the second request (Cache Hit).